# Cross Patform Application Development

AJAX • JSON • Fetch API • jQuery

A Complete Beginner's Step-by-Step Guide

| JSON | AJAX | Fetch API | jQuery |
| --- | --- | --- | --- |
| Data format | Async requests | Modern HTTP | DOM simplifier |

**Learning Objectives — By the end of this lesson, you will be able to:**

- [x] Understand what JSON is and how to read/write it
- [x] Explain what AJAX is and why it matters for modern web pages
- [x] Use the Fetch API to request data from a server
- [x] Use jQuery to simplify DOM manipulation and AJAX calls
- [x] Build a small working project that pulls live data from a public API

# PART 1 — JSON

JavaScript Object Notation • ~12 minutes

## 1.1 What is JSON?

JSON stands for **JavaScript Object Notation**. It is a lightweight **text-based format** used to store and exchange data. Think of it as a way to represent information so that both humans and computers can read it easily.

JSON is the **most popular data format** used on the internet today. When your browser talks to a server (e.g., Facebook, YouTube, Google Maps), they exchange data as JSON.

**Real-World Analogy**

Imagine JSON as a digital form you fill in:
Name: John  Age: 25  City: Kuala Lumpur
JSON puts this information in a structured way that any programming language can understand.

## 1.2 JSON Syntax Rules

Before writing JSON, learn these 5 simple rules:

1. **Curly braces { }** hold an object (a thing with properties)
2. **Square brackets [ ]** hold an array (a list of items)
3. **Keys** must be strings wrapped in **double quotes**
4. **Values** can be: string, number, boolean, null, array, or another object
5. **Key-value pairs** are separated by commas (no trailing comma after the last one)

## 1.3 Your First JSON — A Student Object

**Example: Student JSON Object**

```
{
  "name": "Ahmad Hafiz",
  "age": 22,
  "isEnrolled": true,
  "gpa": 3.8,
  "subjects": ["Math", "Physics", "Programming"],
  "address": {
    "city": "Johor Bahru",
    "country": "Malaysia"
  }
}
```

| Key | Value | Data Type |
|---|---|---|

| | | |
|---|---|---|
| `name` | `"Ahmad Hafiz"` | String — always in double quotes |
| `age` | `22` | Number — no quotes needed |
| `isEnrolled` | `true` | Boolean — true or false (lowercase) |
| `gpa` | `3.8` | Number — decimals are fine |
| `subjects` | `["Math","Physics","Programming"]` | Array — list of values |
| `address` | `{ ... }` | Nested Object — object inside object |

## 1.4 JSON Array — Multiple Students

**Example: Array of Students**

```
[
  { "name": "Ahmad", "age": 22, "grade": "A" },
  { "name": "Siti",  "age": 21, "grade": "B" },
  { "name": "Raj",   "age": 23, "grade": "A+" }
]
```

An array starts with `[` and ends with `]`. Each object inside is separated by a comma.

## 1.5 Working with JSON in JavaScript

**Two key methods you must know:**

- `JSON.parse(text)` → Converts a JSON string into a JavaScript object
- `JSON.stringify(obj)` → Converts a JavaScript object into a JSON string

**Practice: JSON.parse() and JSON.stringify()**

```
// ── JSON.parse() ──────────────────────────────
// Imagine this string came from a server:
const jsonString = '{"name":"Ahmad","age":22}';

// Convert it to a JavaScript object:
const student = JSON.parse(jsonString);

console.log(student.name);  // Output: Ahmad
console.log(student.age);   // Output: 22

// ── JSON.stringify() ──────────────────────────
const myObject = { name: "Siti", age: 21 };

// Convert it to a JSON string (to send to server):
const result = JSON.stringify(myObject);

console.log(result);  // Output: {"name":"Siti","age":21}

// Pretty-print with indentation (for readability):
console.log(JSON.stringify(myObject, null, 2));
```

### ⚠ Common JSON Mistakes (Beginners)

✗ Using single quotes: { 'name': 'Ahmad' } ← WRONG
✓ Always use double quotes: { "name": "Ahmad" } ← CORRECT

✗ Trailing comma: { "name": "Ahmad", } ← WRONG (the last comma breaks JSON)
✓ No trailing comma: { "name": "Ahmad" } ← CORRECT

✗ Undefined or functions — JSON does NOT support them

## 1.6 Try It Yourself

### Exercise 1 — Write Your Own JSON

Open a text editor (Notepad or VS Code) and write a JSON object about yourself with:

- Your name (string)
- Your age (number)
- Your hobbies (array of at least 3 items)
- Your address as a nested object (city, country)

Then open your browser Console (F12 → Console) and:

1. Paste your JSON into JSON.parse() to verify it is valid
2. Use JSON.stringify() to convert a JS object back to string

# PART 2 — AJAX

Asynchronous JavaScript and XML • ~15 minutes

## 2.1 What is AJAX?

**AJAX** stands for **Asynchronous JavaScript And XML**. It is a technique that allows a web page to **communicate with a server in the background** without reloading the entire page.

**Real-World Analogy**

Imagine you are at a restaurant. Old way: every time you order something new, you leave, go home, and come back with a brand new menu. That is very slow!

AJAX way: you stay at the table, the waiter brings you just what you ordered.
The table (page) stays. Only the data changes. That is AJAX!

## 2.2 Without AJAX vs With AJAX

**WITHOUT AJAX**

- User clicks a link
- Browser sends full request to server
- Server sends back an ENTIRE new HTML page
- Browser reloads everything
- User sees a flash/blank screen
- Slow and poor user experience

**WITH AJAX**

- User clicks a button
- JavaScript sends a small request in the background
- Server sends back only the needed data (JSON)
- JavaScript updates just part of the page
- No reload, no flash
- Fast and smooth user experience

## 2.3 The XMLHttpRequest Object (Classic AJAX)

Before modern tools existed, developers used `XMLHttpRequest` (XHR). Though old, it's important to understand the foundation.

**Classic AJAX with XMLHttpRequest**

```
// Step 1: Create the XMLHttpRequest object
const xhr = new XMLHttpRequest();

// Step 2: Configure the request
//  Method: "GET" (fetching data)
//  URL: the address to get data from
```

```
//  true: make it asynchronous (non-blocking)
xhr.open("GET", "https://jsonplaceholder.typicode.com/users/1", true);

// Step 3: Define what happens when we receive a response
xhr.onload = function() {
  if (xhr.status === 200) {            // 200 = OK
    const data = JSON.parse(xhr.responseText);
    console.log("Name:", data.name);
    console.log("Email:", data.email);
  } else {
    console.log("Error! Status:", xhr.status);
  }
};

// Step 4: Handle network errors
xhr.onerror = function() {
  console.log("Network error occurred");
};

// Step 5: Send the request
xhr.send();
```

## 2.4 Understanding Asynchronous Code

This is the most important concept to understand. **Asynchronous** means the code does NOT wait for the request to finish. It continues running other code and comes back when the data is ready.

**Demo: Asynchronous Execution Order**

```
console.log("1. Code starts");

const xhr = new XMLHttpRequest();
xhr.open("GET", "https://jsonplaceholder.typicode.com/posts/1", true);

xhr.onload = function() {
  console.log("3. Data received from server!");  // runs LAST
  const post = JSON.parse(xhr.responseText);
  console.log("   Title:", post.title);
};

xhr.send();

console.log("2. This runs BEFORE the data arrives");

// Output order:
// 1. Code starts
// 2. This runs BEFORE the data arrives
// 3. Data received from server!   ← arrives later
```

**Key Concept: HTTP Status Codes**

When a server responds, it includes a status code. These are the most important ones:
200 → OK (request succeeded)
201 → Created (new resource was created)
400 → Bad Request (something wrong with your request)
401 → Unauthorized (you need to log in first)
403 → Forbidden (you don't have permission)
404 → Not Found (the resource doesn't exist)
500 → Internal Server Error (server had a problem)

## 2.5 AJAX with POST — Sending Data

So far we used `GET` to **receive** data. Use `POST` to **send** data to the server.

**AJAX POST Request — Sending Data**

```
const xhr = new XMLHttpRequest();
xhr.open("POST", "https://jsonplaceholder.typicode.com/posts", true);

// IMPORTANT: Tell the server the format of data we are sending
xhr.setRequestHeader("Content-Type", "application/json");

xhr.onload = function() {
  if (xhr.status === 201) {   // 201 = Created
    const response = JSON.parse(xhr.responseText);
    console.log("Post created with ID:", response.id);
  }
};

// Data to send (must convert object → JSON string)
const newPost = {
  title: "My First Post",
  body: "Hello World from AJAX!",
  userId: 1
};

xhr.send(JSON.stringify(newPost));   // send the JSON string
```

**Exercise 2 — Classic AJAX**

1. Open a new HTML file in VS Code
2. Add a <button> with id='loadBtn' and a  with id='result'
3. Write JavaScript that, when the button is clicked:
   a. Creates an XMLHttpRequest
   b. Fetches https://jsonplaceholder.typicode.com/users/1
   c. Parses the JSON response
   d. Displays the user's name and email inside the 
4. Open the browser and test it!

# 🔗 PART 3 — Fetch API

Modern Asynchronous HTTP Requests • ~18 minutes

## 3.1 Why Fetch API?

The **Fetch API** is the **modern, cleaner** way to make HTTP requests in JavaScript. It replaces XMLHttpRequest and is built into all modern browsers. It uses **Promises** to handle asynchronous operations.

**XMLHttpRequest**

- Verbose and complex
- Callback-based (nested, hard to read)
- Error handling is messy
- Old (from 1999)

**Fetch API**

- Clean and readable
- Promise-based (.then chains)
- Better error handling
- Modern standard

## 3.2 Understanding Promises

A **Promise** represents a value that will be available **in the future**. Think of it as ordering food online — you get a **promise** that your food will arrive. The food is not here yet, but you can plan what to do when it arrives (`.then`) or if it fails (`.catch`).

**Understanding Promises**

```
// A Promise has 3 states:
// 1. Pending   → waiting (request is in progress)
// 2. Fulfilled → success (data received)
// 3. Rejected  → failure (error occurred)

// Basic Promise structure:
somePromise
  .then(function(result) {
    // This runs when the Promise succeeds
    console.log("Success:", result);
  })
  .catch(function(error) {
    // This runs if the Promise fails
    console.log("Error:", error);
  });
```

## 3.3 Your First Fetch Request

**Basic Fetch GET Request**

```
// fetch() returns a Promise
fetch("https://jsonplaceholder.typicode.com/users/1")

  // First .then() converts the Response to JSON
  .then(function(response) {
    console.log("Response status:", response.status);  // 200
    return response.json();  // This also returns a Promise!
  })

  // Second .then() works with the actual data
  .then(function(data) {
    console.log("User name:", data.name);
    console.log("Email:",     data.email);
    console.log("Phone:",     data.phone);
  })

  // .catch() handles any errors
  .catch(function(error) {
    console.log("Something went wrong:", error);
  });
```

> ⚠ **Important: Two .then() Calls**
>
> Why do we need TWO .then() calls with Fetch?
>
> 1st .then() → We get the Response object (headers, status, etc.)
> We must call .json() to read the body
> .json() is also async, so it returns another Promise
>
> 2nd .then() → We finally have the actual data (the parsed JSON object)
>
> This is the most common confusion for beginners — always remember the two steps!

## 3.4 Modern Approach: async / await

The **async/await** syntax is the most modern and **readable** way to work with Promises. Instead of chaining `.then()`, you write code that **looks synchronous** but actually runs asynchronously.

**Fetch with async/await (Recommended Style)**

```
// async keyword makes this function return a Promise
async function getUser(id) {
  try {
    // await pauses execution until the Promise resolves
    const response = await
fetch(`https://jsonplaceholder.typicode.com/users/${id}`);

    // Check if request was successful
    if (!response.ok) {
      throw new Error(`HTTP Error! Status: ${response.status}`);
```

```
    }

    // await the .json() conversion too
    const user = await response.json();

    console.log("Name:",    user.name);
    console.log("Email:",   user.email);
    console.log("Website:", user.website);

  } catch (error) {
    // try/catch replaces .catch()
    console.log("Error:", error.message);
  }
}

// Call the function:
getUser(1);
getUser(5);
```

## 3.5 Fetch POST — Sending Data

**Fetch POST Request**

```
async function createPost() {
  const newPost = {
    title:  "Learning Fetch API",
    body:   "Fetch makes AJAX much easier!",
    userId: 1
  };

  try {
    const response = await fetch("https://jsonplaceholder.typicode.com/posts", {
      method:  "POST",          // HTTP method
      headers: {
        "Content-Type": "application/json",   // Tell server: we are sending JSON
      },
      body: JSON.stringify(newPost)            // Convert object to JSON string
    });

    const data = await response.json();
    console.log("Created post ID:", data.id);
    console.log("Title:", data.title);

  } catch (error) {
    console.log("Failed to create post:", error);
  }
}

createPost();
```

# 3.6 Full Project — User Card Display

**Let's build a real mini-project** that fetches a list of users and displays them as cards on a webpage.

**🌐 Full Project: User Card Display (copy and save as index.html)**

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>User Cards — Fetch API Project</title>
  <style>
    body { font-family: Arial, sans-serif; background: #f0f4f8; padding: 20px; }
    h1   { color: #1F4E79; text-align: center; }
    .grid{ display: flex; flex-wrap: wrap; gap: 16px; justify-content: center; }
    .card{ background: white; border-radius: 8px; padding: 20px;
           width: 250px; box-shadow: 0 2px 8px rgba(0,0,0,0.1); }
    .card h3 { color: #2E75B6; margin: 0 0 8px; }
    .card p  { color: #555; font-size: 14px; margin: 4px 0; }
    button   { display: block; margin: 20px auto; padding: 12px 28px;
               background: #2E75B6; color: white; border: none;
               border-radius: 6px; font-size: 16px; cursor: pointer; }
    button:hover { background: #1F4E79; }
    #status { text-align: center; color: #888; font-style: italic; }
  </style>
</head>
<body>
  <h1>👥 User Directory</h1>
  <button onclick="loadUsers()">Load Users</button>
  <p id="status">Click the button to load users</p>
  <div class="grid" id="userGrid"></div>

  <script>
    async function loadUsers() {
      const status = document.getElementById("status");
      const grid   = document.getElementById("userGrid");

      status.textContent = "Loading...";
      grid.innerHTML = "";

      try {
        const response = await
fetch("https://jsonplaceholder.typicode.com/users");

        if (!response.ok) throw new Error("Failed to fetch users");

        const users = await response.json();
        status.textContent = `Loaded ${users.length} users`;

        users.forEach(user => {
          const card = document.createElement("div");
          card.className = "card";
          card.innerHTML = `
            <h3>${user.name}</h3>
            <p>📧 ${user.email}</p>
            <p>📞 ${user.phone}</p>
            <p>🌐 ${user.website}</p>
            <p>🏙️ ${user.address.city}</p>
```

```
          `;
          grid.appendChild(card);
        });

      } catch (error) {
        status.textContent = "Error: " + error.message;
        status.style.color = "red";
      }
    }
  </script>
</body>
</html>
```

### Exercise 3 — Fetch API Challenge

1. Copy the User Card project above and save it as 'users.html'
2. Open it in your browser and click Load Users
3. CHALLENGE A: Add a second button 'Load Posts' that fetches:
   https://jsonplaceholder.typicode.com/posts
   Show the first 5 posts (title + body)
4. CHALLENGE B: Add an input field where you type a user ID (1–10)
   and a button that loads only THAT user's details

# PART 4 — jQuery

Write Less, Do More • ~15 minutes

## 4.1 What is jQuery?

**jQuery** is a fast, small, and feature-rich **JavaScript library**. It makes HTML document traversal, DOM manipulation, event handling, and AJAX much simpler. The motto is: **"Write Less, Do More."**

Though modern JavaScript (Fetch API, querySelector) has reduced the need for jQuery, it is still widely used in many websites and is important to know.

## 4.2 Including jQuery

Add this `<script>` tag inside the `<head>` or just before `</body>`:

**Including jQuery via CDN**

```
<!-- Option 1: CDN (recommended for learning — no installation) -->
<script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>

<!-- Always put YOUR script after jQuery -->
<script src="your-script.js"></script>
```

## 4.3 The $ Sign and Selectors

In jQuery, **$ is the jQuery function**. You use it to **select HTML elements**. The syntax mirrors CSS selectors — if you know CSS, you already know jQuery selectors!

| jQuery Selector | What It Selects | CSS Equivalent |
|---|---|---|
| `$("#myDiv")` | Element with id="myDiv" | #myDiv |
| `$(".card")` | All elements with class="card" | .card |
| `$("p")` | All <p> elements | p |
| `$("ul li")` | All <li> inside <ul> | ul li |
| `$("input[type=text]")` | Text input fields | input[type=text] |
| `$(this)` | The current element | (no direct equiv.) |

## 4.4 jQuery vs Plain JavaScript — Side by Side

**jQuery vs Plain JavaScript Comparison**

```
// ── HIDE an element ──────────────────────────────
```

```
document.getElementById("box").style.display = "none";  // Plain JS
$("#box").hide();                                        // jQuery ✓

// ── CHANGE text content ──────────────────────────
document.getElementById("msg").textContent = "Hello!";  // Plain JS
$("#msg").text("Hello!");                                // jQuery ✓

// ── ADD a CSS class ──────────────────────────────
document.getElementById("btn").classList.add("active"); // Plain JS
$("#btn").addClass("active");                            // jQuery ✓

// ── HANDLE a click event ─────────────────────────
document.getElementById("btn").addEventListener("click", function() {  // Plain JS
  console.log("clicked!");
});
$("#btn").click(function() {                             // jQuery ✓
  console.log("clicked!");
});
```

## 4.5 Common jQuery Methods

**Essential jQuery Methods**

```
// ── Text and HTML ────────────────────────────────
$("#title").text();               // Get text content
$("#title").text("New Title");    // Set text content
$("#box").html();                 // Get HTML content
$("#box").html("<b>Bold!</b>");   // Set HTML content

// ── CSS and Classes ──────────────────────────────
$("#box").css("color", "red");              // Set one CSS property
$("#box").css({ color:"red", fontSize:"18px" }); // Set multiple
$("#box").addClass("highlight");            // Add class
$("#box").removeClass("highlight");         // Remove class
$("#box").toggleClass("highlight");         // Toggle on/off

// ── Show / Hide ──────────────────────────────────
$("#box").hide();                 // Hide (display:none)
$("#box").show();                 // Show
$("#box").toggle();               // Toggle show/hide
$("#box").fadeIn(500);            // Fade in (500ms)
$("#box").fadeOut(500);           // Fade out
$("#box").slideDown(300);         // Slide down
$("#box").slideUp(300);           // Slide up

// ── Form Values ──────────────────────────────────
$("#nameInput").val();            // Get input value
$("#nameInput").val("John");      // Set input value
```

## 4.6 jQuery Events

**jQuery Event Handling**

```
// ── Click event ──────────────────────────────────
$("#submitBtn").click(function() {
  const name = $("#nameInput").val();
  $("#output").text("Hello, " + name + "!");
});

// ── Hover event ──────────────────────────────────
$(".card").hover(
  function() { $(this).css("box-shadow", "0 4px 20px rgba(0,0,0,0.2)"); },
  function() { $(this).css("box-shadow", "none"); }
);

// ── Keypress event ───────────────────────────────
$("#searchBox").keyup(function() {
  const query = $(this).val();
  console.log("Searching for:", query);
});

// ── Document ready ───────────────────────────────
// IMPORTANT: Wrap your code in this to ensure
// DOM is fully loaded before your script runs
$(document).ready(function() {
  console.log("Page is ready!");
  // All your jQuery code goes here
});

// Shorter version (same thing):
$(function() {
  console.log("Page is ready!");
});
```

## 4.7 jQuery AJAX — $.ajax() and $.get()

jQuery also has its own **built-in AJAX methods** that simplify HTTP requests.

**jQuery AJAX Methods**

```
// ── $.get() — Simple GET request ─────────────────
$.get("https://jsonplaceholder.typicode.com/users/1", function(data) {
  console.log("Name:", data.name);
  console.log("Email:", data.email);
});


// ── $.ajax() — Full control ──────────────────────
$.ajax({
  url:      "https://jsonplaceholder.typicode.com/posts",
  method:   "POST",
  dataType: "json",
  data: JSON.stringify({
    title:  "jQuery AJAX Post",
```

```
    body:   "Created with jQuery!",
    userId: 1
  }),
  contentType: "application/json",

  success: function(response) {
    console.log("Post created! ID:", response.id);
  },

  error: function(xhr, status, error) {
    console.log("Error:", error);
  }
});
```

## 4.8 Full jQuery Project — Live Search

Let's build a **live user search** using jQuery and the Fetch API together:

**Full jQuery Project: Live User Search (save as search.html)**

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>jQuery Live Search</title>
  <script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
  <style>
    body   { font-family:Arial,sans-serif; max-width:600px; margin:40px auto;
padding:0 20px; }
    h1     { color:#1F4E79; }
    input  { width:100%; padding:12px; font-size:16px; border:2px solid #ccc;
             border-radius:6px; box-sizing:border-box; }
    input:focus { border-color:#2E75B6; outline:none; }
    .result{ background:white; border:1px solid #ddd; border-radius:6px;
             padding:14px; margin-top:10px; }
    .result h3 { margin:0 0 6px; color:#2E75B6; }
    .result p  { margin:2px 0; color:#555; font-size:14px; }
    #status    { color:#888; font-style:italic; margin-top:10px; }
  </style>
</head>
<body>
  <h1>🔍 Live User Search</h1>
  <input type="text" id="searchBox" placeholder="Type a name to search...">
  <p id="status">Loading users...</p>
  <div id="results"></div>

  <script>
    let allUsers = [];   // Store all users once loaded

    // Load all users when page is ready
    $(document).ready(function() {
      $.get("https://jsonplaceholder.typicode.com/users", function(users) {
        allUsers = users;
        $("#status").text("Type to search " + users.length + " users");
```

```
    });
  });

  // Filter users as you type
  $("#searchBox").keyup(function() {
    const query = $(this).val().toLowerCase().trim();
    const resultsDiv = $("#results");
    resultsDiv.empty();

    if (query === "") {
      $("#status").text("Type to search " + allUsers.length + " users");
      return;
    }

    const matches = allUsers.filter(function(user) {
      return user.name.toLowerCase().includes(query) ||
             user.email.toLowerCase().includes(query);
    });

    if (matches.length === 0) {
      $("#status").text("No users found for: " + query);
      return;
    }

    $("#status").text("Found " + matches.length + " user(s)");

    matches.forEach(function(user) {
      const card = $(`
        <div class="result">
          <h3>${user.name}</h3>
          <p>📧 ${user.email}</p>
          <p>📞 ${user.phone}</p>
          <p>🏙️ ${user.address.city}</p>
        </div>
      `);
      resultsDiv.append(card);
    });
  });
</script>
</body>
</html>
```

### 🎯 Exercise 4 — jQuery Challenge

1. Copy the Live Search project above and save as 'search.html'
2. Open in browser and try searching for a name
3. CHALLENGE A: Add a 'Clear' button that empties the search box and results
4. CHALLENGE B: Add a counter that shows how many results are displayed
5. CHALLENGE C: Make the cards fade in using jQuery's .fadeIn() method

# PART 5 — Final Project

Bringing It All Together • ~10 minutes

## 5.1 Final Project: Country Info App

This final project combines **JSON + Fetch API + jQuery** into one complete application. The user types a country name and gets information about it from a public API.

**Final Project: Country Info App (save as country-app.html)**

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>🌍 Country Info App</title>
  <script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
  <style>
    * { box-sizing: border-box; }
    body    { font-family:Arial,sans-serif; background:#EBF3FB;
              min-height:100vh; display:flex; flex-direction:column; align-
items:center; padding:40px 20px; }
    h1      { color:#1F4E79; font-size:2rem; }
    .search { display:flex; gap:10px; width:100%; max-width:500px; margin-
bottom:24px; }
    input   { flex:1; padding:12px 16px; font-size:16px;
              border:2px solid #B0C9E0; border-radius:8px; }
    button  { padding:12px 24px; background:#2E75B6; color:white;
              border:none; border-radius:8px; font-size:16px; cursor:pointer; }
    button:hover { background:#1F4E79; }
    #card   { background:white; border-radius:12px; padding:28px; width:100%;
              max-width:500px; box-shadow:0 4px 16px rgba(0,0,0,0.1);
display:none; }
    #card img    { width:120px; border:2px solid #ddd; border-radius:4px; }
    #card h2     { color:#1F4E79; margin:12px 0 6px; }
    .info-row    { display:flex; justify-content:space-between;
                   padding:8px 0; border-bottom:1px solid #f0f0f0; font-size:15px;
}
    .info-row span:first-child { color:#888; }
    .info-row span:last-child  { font-weight:bold; color:#333; }
    #error  { color:#c0392b; font-weight:bold; }
  </style>
</head>
<body>
  <h1>🌍 Country Info App</h1>
  <div class="search">
    <input type="text" id="countryInput" placeholder="Enter country name...">
    <button id="searchBtn">Search</button>
  </div>
  <p id="error"></p>
  <div id="card">
    <img id="flag" src="" alt="Flag">
    <h2 id="countryName"></h2>
    <div id="info"></div>
```

```
  </div>

  <script>
    async function searchCountry(name) {
      $("#error").text("");
      $("#card").hide();

      try {
        const url =
`https://restcountries.com/v3.1/name/${encodeURIComponent(name)}?fullText=true`;
        const response = await fetch(url);

        if (!response.ok) throw new Error("Country not found. Try: Malaysia,
Japan, Brazil");

        const data    = await response.json();
        const country = data[0];

        // Extract info from JSON
        const capital    = country.capital?.[0]       || "N/A";
        const population = country.population.toLocaleString();
        const region     = country.region;
        const currency   = Object.values(country.currencies || {})[0]?.name ||
"N/A";
        const languages  = Object.values(country.languages  || {}).join(", ");

        // Update DOM using jQuery
        $("#flag").attr("src", country.flags.png);
        $("#countryName").text(country.name.common);

        const rows = [
          ["Capital",    capital],
          ["Population", population],
          ["Region",     region],
          ["Currency",   currency],
          ["Languages",  languages],
        ];

        $("#info").empty();
        rows.forEach(([label, value]) => {
          $("#info").append(`
            <div class="info-row">
              <span>${label}</span>
              <span>${value}</span>
            </div>
          `);
        });

        $("#card").fadeIn(400);

      } catch (err) {
        $("#error").text("❌ " + err.message);
      }
    }

    // Button click
    $("#searchBtn").click(function() {
```

```
      const name = $("#countryInput").val().trim();
      if (name) searchCountry(name);
    });

    // Enter key
    $("#countryInput").keypress(function(e) {
      if (e.which === 13) $("#searchBtn").click();
    });
  </script>
</body>
</html>
```

### What This Project Demonstrates

- ☑ JSON — Parsing complex nested JSON from a real API
- ☑ Fetch API — async/await with error handling
- ☑ jQuery — DOM manipulation: .text(), .attr(), .append(), .fadeIn(), .hide()
- ☑ jQuery Events — .click() and .keypress() event handling
- ☑ Real API — Uses restcountries.com (a free, public REST API)

# Quick Reference Cheatsheet

Keep this handy while coding!

## JSON Cheatsheet

**JSON**

```
JSON.parse('{"name":"Ali"}')  → { name: "Ali" }   // String → Object
JSON.stringify({ name:"Ali" }) → '{"name":"Ali"}'  // Object → String

// Valid value types:   string, number, boolean, null, array, object
// Keys: MUST be double-quoted strings
// No trailing commas, no functions, no undefined
```

## Fetch API Cheatsheet

**Fetch API**

```
// GET
const res  = await fetch(url);
const data = await res.json();

// POST
const res = await fetch(url, {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify(data)
});
```

## jQuery Cheatsheet

**jQuery**

```
// Select        $("#id")  $(".class")  $("tag")
// Text          .text()  .text("new")
// HTML          .html()  .html("<b>new</b>")
// CSS           .css("prop","val")  .addClass("x")  .removeClass("x")
// Show/Hide     .show()  .hide()  .toggle()  .fadeIn()  .fadeOut()
// Events        .click(fn)  .keyup(fn)  .hover(fn,fn)
// DOM           .append(html)  .prepend(html)  .empty()  .remove()
// Attributes    .attr("src","url")  .val()  .val("new")
// AJAX          $.get(url, fn)  $.ajax({ url, method, data, success, error })
```

## Topic Comparison

| | JSON | AJAX | Fetch API | jQuery |
|---|---|---|---|---|

| | | | | |
|---|---|---|---|---|
| **Type** | Data format | Technique | Browser API | JS Library |
| **Purpose** | Store data | Async communication | HTTP requests | Simplify JS |
| **Syntax** | Text-based | XHR object | fetch() + Promises | $ functions |
| **Year** | 2001 | 1999 | 2015 | 2006 |
| **Use today?** | ☑ Always | ☑ Concept | ☑ Yes (preferred) | ☑ Many sites |

---

## Congratulations! You have completed the AJAX · JSON · Fetch API · jQuery lesson.

Keep practising with the free API at: jsonplaceholder.typicode.com • restcountries.com • openweathermap.org